

ESPRIT SCHOOL OF ENGINEERING

www.esprit.tn - E-mail : contact@esprit.tn

Siége Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : 1-2 rue André Ampère - 2083 - Pôle Technologique - El Ghazala - Tél +216 70 250 000 - Fax +216 70 685454

esprit
Se former autrement

HONORIS UNITED UNIVERSITIES

2024 / 2025

SPÉCIALITÉ : DATA SCIENCE

Détection de textes générés par IA (type ChatGPT) vs textes humains

Réalisé par: TLILI Mohamed Aziz

Encadrant ESPRIT: Mme ENNINE Hanene

EUR-ACE® Délivrée par la Commission des Titres d'Ingénieur

CONFÉRENCE DES GRANDES ÉCOLES

cdio

AUF Agence Universitaire de la Francophonie

IFGU

Dédicaces

Je dédie ce projet

À mes parents Nouredine TLILI et Imen DRIDI

Qui m'ont soutenu et motivé tout au long de ce parcours.
Ce travail vous est entièrement dédié,
et j'espère vous rendre fiers.

À mon encadrante académique Mme Hanene Ennine

Pour sa disponibilité, ses précieux conseils
et son accompagnement bienveillant tout au long de ce projet.
Sans son aide, ce travail n'aurait pas pu aboutir.

Je vous remercie pour vos encouragements et votre soutien
tout au long de cette période.

Table des matières

Introduction Générale **1**

1 Contexte général **2**
1.1 Introduction 2
1.2 Présentation de l'organisme d'accueil 2
1.3 Cadre stratégique et valeurs de l'organisme 2
1.4 Méthodologie CRISP-DM 3
1.5 Les étapes de la méthodologie CRISP-DM 3
1.6 Conclusion 4

2 Compréhension métier **5**
2.1 Problématique 5
2.2 Etude de l'existant 5
2.3 Solution Proposée 6
2.4 Conclusion 6

3 Compréhension et Préparation des Données **7**
3.1 Collecte de données 7
3.2 Description des données 8
3.2.1 Informations sur le Donneés 8
3.2.2 Analyse statistique 8
3.3 Exploration des données 9
3.4 Préparation des Donneés 12
3.5 Conclusion 13

4 Modélisation **14**
4.1 Modéle 1 : Long Short-Term Memory (LSTM) 14
4.1.1 Définition 14
4.1.2 Architecture 14
4.1.3 Fonctionnement 15
4.1.4 Compositions 15
4.2 Modéle 2 : Convolutional Neural Network (CNN) 16
4.2.1 Définition 16
4.2.2 Architecture 16
4.2.3 Fonctionnement 17
4.2.4 Conclusion 17

5 Évaluation du modèle **18**
5.1 Introduction 18
5.2 Métriques d'évaluation 18
5.3 Résultats expérimentaux 19
5.3.1 Résultats du modèle LSTM 19
5.3.2 Résultats du modèle CNN 19
5.3.3 Comparaison des deux modèles 19
5.4 Analyse des résultats 20
5.4.1 Analyse du modèle LSTM 20
5.4.2 Analyse du modèle CNN 20
5.4.3 Comparaison de deux modéles 20
5.5 Conclusion 20

6 Déploiement **21**
6.1 Introduction 21
6.2 Technologies utilisées 21
6.2.1 FastAPI 21
6.2.2 React et Vite 21
6.2.3 Render 21
6.2.4 Git et GitHub 21
6.3 Déploiement sur Render 22
6.3.1 Étapes de déploiement 22
6.3.2 Résultat 22
6.4 Tâches réalisées 22
6.5 Conclusion 23

Conclusion Générale **24**

Table des figures

1.1 Logo d'Esprit-Tech 2
1.2 Processus CRISP-DM 4

3.1 Dataset pour la détection des textes IA vs humains. 7
3.2 Informations sur le Dataset. 8
3.3 Analyse statistique de la variable `generated` 8
3.4 Distribution des classes de la variable cible generated 9
3.5 Distribution des classes de la variable cible generated aprés le sous-échantillonnage 10
3.6 Distribution du nombre de tokens dans les textes 10
3.7 Comparaison des tokens les plus fréquents pour les textes humains et IA . 11
3.8 Comparaison des 3-gramms les plus fréquents pour les textes humains et IA 12

4.1 Architecture du modéle LSTM 15
4.2 Architecture du modéle CNN 16

6.1 Résultat *Human Written* — texte classifié comme rédigé par un humain. . 23
6.2 Résultat *AI Generated* — texte classifié comme généré par une IA. 23

Liste des tableaux

2.1 Solutions existantes pour la détection de textes générés par IA 5

5.1 Rapport de classification – Modèle LSTM 19
5.2 Rapport de classification – Modèle CNN 19
5.3 Comparaison des performances LSTM vs CNN 20

6.1 Routes de l'API FastAPI 21

Introduction Générale

Aujourd'hui, l'intelligence artificielle (IA) se développe très rapidement, notamment dans le domaine du traitement automatique du langage naturel (NLP). Des outils comme ChatGPT sont désormais capables de produire des textes difficilement distinguables de ceux rédigés par des humains.

Cette évolution soulève des problèmes importants dans plusieurs domaines : la triche dans l'enseignement, la désinformation dans les médias et la modération des contenus sur Internet. Il devient donc indispensable de disposer d'outils fiables pour détecter automatiquement l'origine d'un texte.

Ce projet a pour objectif de concevoir un système de classification automatique des textes humains et générés par IA, en utilisant des techniques de Deep Learning. Nous avons suivi la méthodologie CRISP-DM, qui structure le travail en plusieurs phases : la compréhension métier, la préparation des données, la modélisation, l'évaluation et le déploiement.

Chapitre 1

Contexte général

1.1 Introduction

Dans ce chapitre, nous présentons l'organisme d'accueil et le cadre dans lequel le projet a été réalisé. Nous décrivons ensuite le contexte général du projet, en soulignant les enjeux liés à la détection des textes générés par l'intelligence artificielle. Enfin, nous expliquons la méthodologie choisie, basée sur l'approche CRISP-DM, pour structurer et guider l'ensemble du projet.

1.2 Présentation de l'organisme d'accueil

Créée en 2010, ESPRIT-Tech est la structure dédiée à la Recherche, Développement et Innovation (RDI) au sein de l'école ESPRIT. Depuis la mise en place des programmes de formation, l'activité RDI est devenue l'une des priorités stratégiques de l'établissement. ESPRIT-Tech regroupe des enseignants-chercheurs et offre un soutien complet en matière de financement, de formation et d'accompagnement dans les projets de recherche. La structure définit les stratégies de recherche en collaboration avec les différentes parties prenantes, en mettant particulièrement l'accent sur l'innovation et la recherche appliquée.



FIGURE 1.1 – Logo d'Esprit-Tech

1.3 Cadre stratégique et valeurs de l'organisme

ESPRIT-Tech a pour vision de promouvoir la recherche, l'innovation et la valorisation des résultats scientifiques. Elle s'engage à soutenir l'excellence en RDI à travers le développement de projets à fort impact, répondant aux besoins académiques et industriels.
Sa mission consiste à créer un environnement propice à la recherche et à l'innovation, à

renforcer le financement externe et à développer un écosystème entrepreneurial ouvert à l'international, favorisant la collaboration entre chercheurs et partenaires.
Les valeurs d'ESPRIT-Tech reposent sur la rigueur scientifique, le respect, l'inclusion et la transparence. Sa démarche s'inscrit dans une logique de coopération, de responsabilité sociale et de développement durable.

1.4 Méthodologie CRISP-DM

CRISP-DM ou Cross-Industry Standard Process for Data Mining, est un modèle de processus qui permet d'orienter les travaux d'exploration de données en data science. Cette méthodologie itérative, utilisée par nos data scientist, permet de travailler en cycles : du business understanding pour comprendre le besoin client jusqu'à l'évaluation où l'équipe présente les résultats aux clients et le déploiement. Cette méthode permet de donner un cadre structuré pour guider les data scientist tout au long du cycle de vie d'un projet.

1.5 Les étapes de la méthodologie CRISP-DM

La méthodologie CRISP-DM se compose de 6 étapes clés :

Etape 1 : La compréhension du besoin client
Dans cette phase initiale, l'équipe de data science travaille en collaboration avec les parties prenantes afin de comprendre les objectifs commerciaux, les exigences, les contraintes du projet ainsi que les bénéfices attendus. Les ressources nécessaires pour réaliser le projet, tels que le budget, les compétences techniques et l'accès aux données, sont évalués. Les problèmes à résoudre sont identifiés et les critères permettant de mesurer le succès du projet sont définis. Cette étape est essentielle pour garantir la bonne atteinte des objectifs du projet.

Etape 2 : La compréhension des données
Cette phase consiste à collecter, explorer et évaluer toutes les données disponibles pour le projet. L'équipe de data science analyse ces dernières pour comprendre leur structure, leur qualité, leurs éventuels problèmes, leur pertinence et leur disponibilité. Cela permet d'identifier les valeurs manquantes, ou les erreurs qui pourraient affecter les analyses suivantes. Elle se concerte ensuite avec les experts métiers pour identifier des pistes de résolution des problèmes constatés et ainsi aider à interpréter les données.

Etape 3 : La préparation des données
Une fois que les données ont été évaluées, il faut à présent sélectionner les variables et les échantillons pertinents pour l'analyse, en fonction des objectifs qui ont été fixés. Les données sont ensuite nettoyées et, si nécessaire, transformées. Cela peut inclure la gestion des valeurs manquantes, l'échantillonnage des données et la création de variables dérivées. Si plusieurs sources de données sont utilisées, elles sont intégrées pour créer un ensemble de données cohérent. Cette étape monopolise plus de la moitié du temps sur l'ensemble du projet.

Etape 4 : La modélisation
Dans cette phase, à partir de techniques d'analyse des données, l'équipe de data science

construit des modèles prédictifs ou des modèles descriptifs, en fonction des objectifs du projet. Plusieurs itérations ont lieu entre les étapes de préparation et de modélisation pour affiner l'utilisation de certains algorithmes particuliers. L'étape du modeling génère souvent plusieurs modèles de Data Mining qui répondent tous à la même problématique.

Etape 5 : L'évaluation
Une fois que les modèles ont été construits, ils sont évalués pour déterminer leur qualité et leur précision. Cette étape du cycle permet de s'assurer que le modèle permet d'atteindre les objectifs du projet. Les performances des modèles sont mesurées à l'aide de métriques appropriées et comparées aux critères de succès définis dans la première phase du cycle. Si les résultats ne répondent pas aux attentes ou s'il y a des problèmes identifiés, les étapes précédentes peuvent être révisées et répétées pour améliorer les performances du modèle

Etape 6 : Le déploiement
Enfin, les résultats du projet sont présentés aux parties prenantes et sont intégrés si nécessaire dans les systèmes existants pour aider la prise de décision. Cette phase implique souvent la création de rapports, de visualisations ou d'autres formes de communication pour rendre les résultats compréhensibles et utilisables par ceux qui ne seraient pas spécialistes des données. A cette étape, le modèle est performant et répond correctement à la problématique.

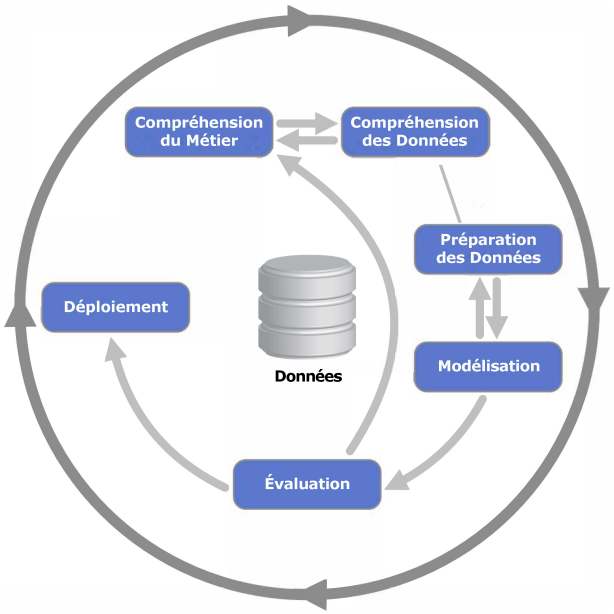


FIGURE 1.2 – Processus CRISP-DM

1.6 Conclusion

Ce chapitre a présenté ESPRIT-Tech, son rôle, sa vision et ses valeurs, ainsi que le projet de détection de textes IA vs humains, en suivant la méthodologie CRISP-DM pour structurer toutes les étapes du projet. Ces éléments servent de base pour la mise en œuvre et l'évaluation du modèle dans les chapitres suivants.

Chapitre 2

Compréhension métier

2.1 Problématique

Avec le développement des intelligences artificielles capables de générer du texte, comme GPT, il devient difficile de savoir si un texte a été écrit par un humain ou par une machine. Cela crée des problèmes dans plusieurs domaines : à l'école ou à l'université (plagiat ou tricherie), dans les médias (fiabilité des informations), dans la sécurité (désinformation) ou dans la recherche scientifique.
Le problème principal est donc de trouver une méthode fiable pour reconnaître automatiquement l'origine d'un texte, afin d'éviter les abus et de garantir que l'information soit crédible.

2.2 Etude de l'existant

Solution	**Avantages**	**Inconvénients**
GPTZero	Simple à utiliser, conçu spécialement pour la détection de textes IA	Moins fiable pour les textes courts ou réécrits par un humain
OpenAI AI Text Classifier	Optimisé pour les modèles OpenAI, facile d'accès	Nécessite des textes longs (150 mots), peut donner des faux positifs/négatifs
Turnitin (anti-plagiat avec détection IA)	Intégré dans les systèmes éducatifs, combine détection de plagiat et d'IA	Précision limitée pour textes courts ou réécrits, accessible surtout aux institutions
DetectGPT	Très prometteuse pour les textes longs, méthode statistique robuste	Complexe à mettre en œuvre, nécessite modèle génératif et connaissances techniques avancées

Table 2.1 – Solutions existantes pour la détection de textes générés par IA

2.3 Solution Proposée

Pour répondre à cette problématique, Nous proposons d'utiliser le deep learning pour distinguer les textes humains des textes générés par une IA comme ChatGPT. Le projet consiste à collecter et préparer un corpus de textes, puis à entraîner des modèles de classification tels que CNN et LSTM. Les modèles sont ensuite évalués avec des métriques standard et le modèle le plus performant pourra être utilisé pour analyser de nouveaux textes,

2.4 Conclusion

Ce chapitre a présenté la compréhension du métier pour notre projet de détection des textes générés par IA. Nous avons défini la problématique, analysé l'existant et exposé la solution proposée, posant ainsi les bases pour les étapes suivantes du projet.

Chapitre 3

Compréhension et Préparation des Données

3.1 Collecte de données

Pour ce projet, nous avons utilisé un dataset public disponible sur Kaggle, spécialement conçu pour la détection des textes générés par intelligence artificielle. Ce dataset contient deux colonnes : la colonne text, qui contient le texte à analyser, et la colonne generated, qui sert de label indiquant si le texte a été écrit par un humain (0) ou généré par une IA (1).

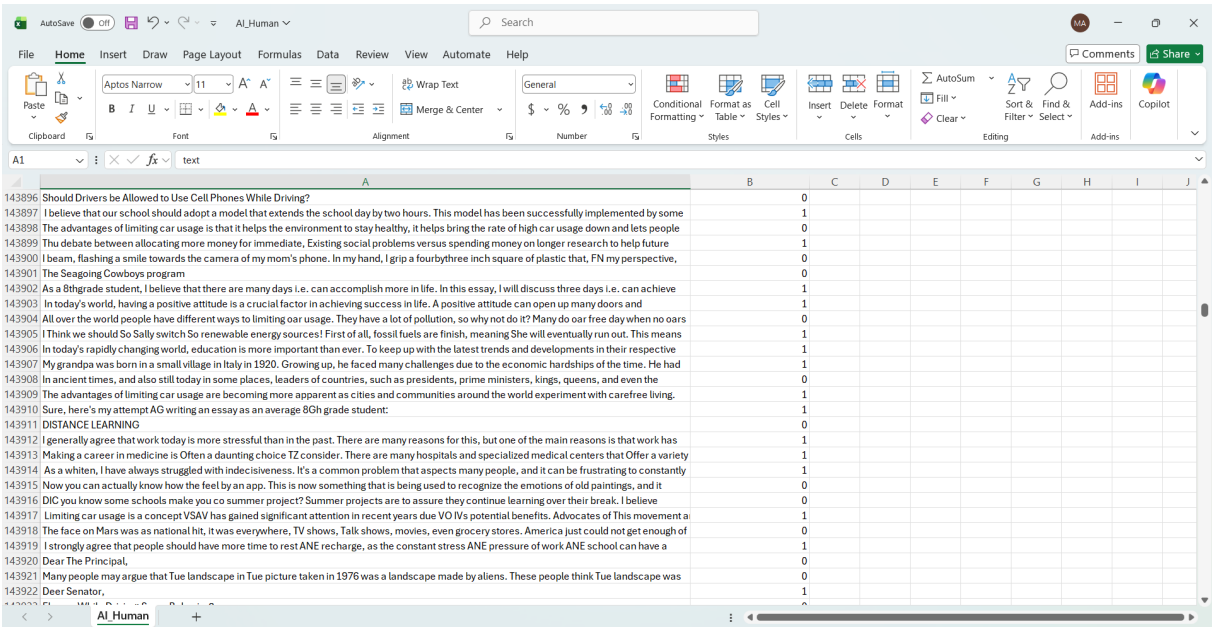


FIGURE 3.1 – Dataset pour la détection des textes IA vs humains.

3.2 Description des données

3.2.1 Informations sur le Donneés

L'image présente un aperçu de la structure et des caractéristiques d'un DataFrame contenant un total de 487 235 entrées.

```
RangeIndex: 487235 entries, 0 to 487234
Data columns (total 2 columns):
 #   Column     Non-Null Count   Dtype
---  ------     --------------   -----
 0   text       487235 non-null  object
 1   generated  487235 non-null  float64
dtypes: float64(1), object(1)
```

FIGURE 3.2 – Informations sur le Dataset.

Le jeu de données est composé de deux colonnes :

text : Cette colonne est de type object, Elle contient 487 235 valeurs non nulles, ce qui indique qu'aucune donnée textuelle n'est manquante dans cette colonne. Il s'agit du contenu textuel à analyser.

generated : Cette colonne est de type float64 et contient 487 235 valeurs non nulles, confirmant l'absence de données manquantes. Comme l'indique son nom, cette variable correspond à l'étiquette binaire de classification : elle prend la valeur 1 pour les textes générés par une intelligence artificielle, et 0 pour les textes rédigés par un humain.

3.2.2 Analyse statistique

```
In [34]: df.describe()
```

Out[34]:

	generated
count	487235.000000
mean	0.372383
std	0.483440
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	1.000000

FIGURE 3.3 – Analyse statistique de la variable `generated`

La fonction `df.describe()` appliquée à la variable `generated` donne les résultats suivants :

- **Count = 487 235** : Le dataset contient 487 235 textes au total.
- **Mean = 0.372** : Comme `generated` est une variable binaire (0 = humain, 1 = IA), cela signifie que 37,2% des textes sont générés par une IA, tandis que 62,8% sont écrits par des humains.
- **Std = 0.483** : L'écart-type proche de 0.5 est normal pour une variable binaire modérément équilibrée.
- **Min = 0 / Max = 1** : La variable ne contient que deux valeurs (0 et 1), ce qui confirme qu'il s'agit d'une classification binaire.
- **25% = 0 / 50% = 0 / 75% = 1** : La médiane égale à 0 indique que plus de la moitié des textes sont humains. Le troisième quartile égal à 1 montre qu'à partir de 75% des observations, on inclut les textes générés par IA.
  La répartition des classes est illustrée par la courbe de distribution ci-dessous :

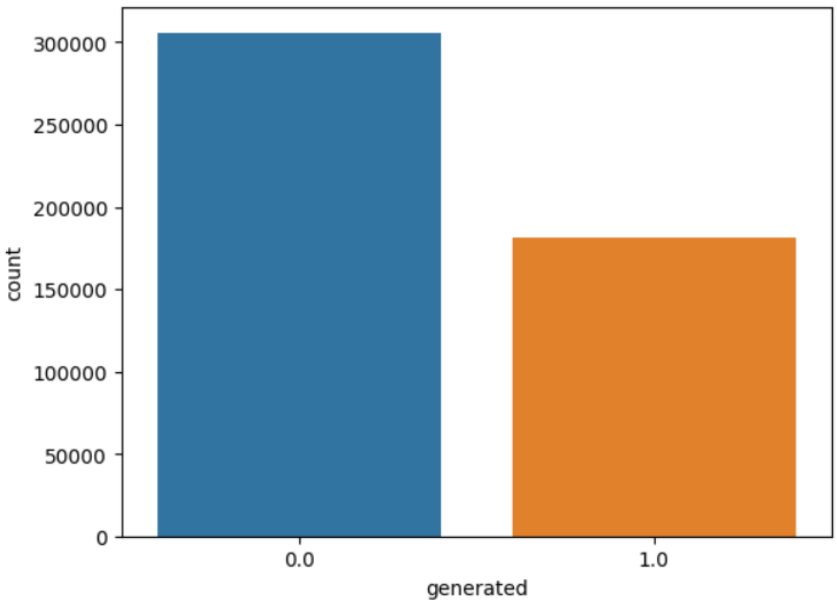


FIGURE 3.4 – Distribution des classes de la variable cible generated

3.3 Exploration des données

Suite à l'observation d'un déséquilibre entre les deux classes lors de l'analyse exploratoire, une étape de rééquilibrage a été effectuée afin de garantir des conditions d'apprentissage optimales pour le modèle de classification. Pour ce faire, nous avons procédé à un sous-échantillonnage aléatoire consistant à sélectionner 50 000 échantillons de la classe majoritaire (textes humains, generated = 0) et 50 000 échantillons de la classe minoritaire (textes IA, generated = 1).

Ce rééquilibrage présente plusieurs avantages :

— Il élimine le biais potentiel du modèle vers la classe majoritaire.

— Il permet d'évaluer les performances de manière équitable sur les deux classes.
— Il facilite l'interprétation des métriques comme la précision et le rappel.

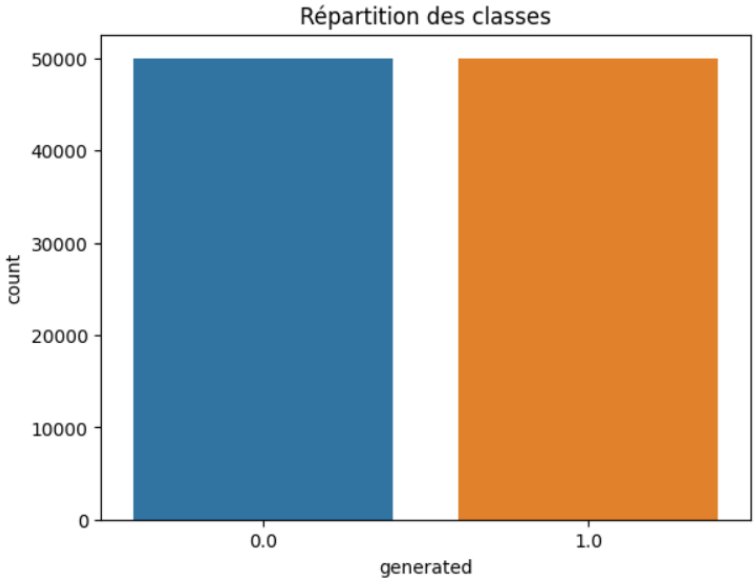


FIGURE 3.5 – Distribution des classes de la variable cible generated aprés le sous-échantillonnage

Avant d'entraîner le modèle, les textes doivent être transformés en une forme numérique compréhensible par le modèle. Cette étape s'appelle la tokenization. La tokenization consiste à découper un texte en petites unités appelées tokens. Un token peut être :

- un mot complet
- une partie de mot
- un symbole ou un signe de ponctuation

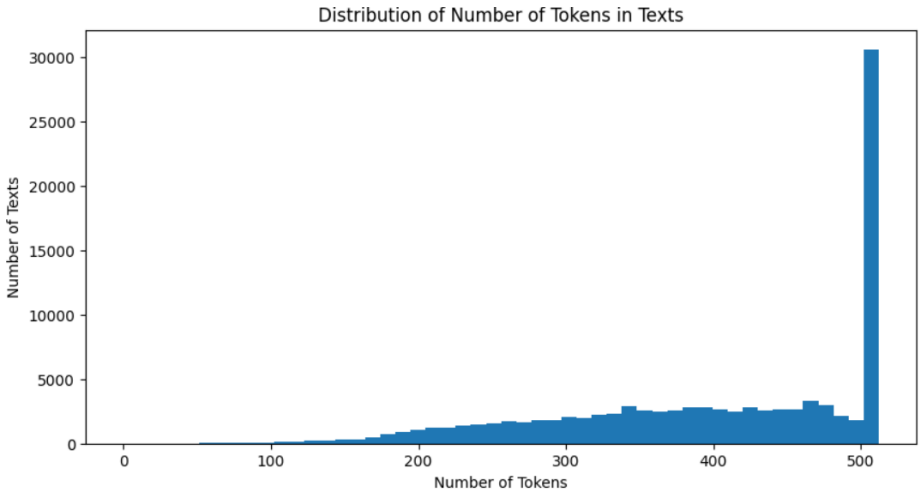


FIGURE 3.6 – Distribution du nombre de tokens dans les textes

Ce graphique montre la distribution du nombre de tokens par texte après la tokenization. Le pic important observé à 512 tokens indique que beaucoup de textes atteignent la limite maximale. Cela signifie que Une grande partie des textes sont longs.

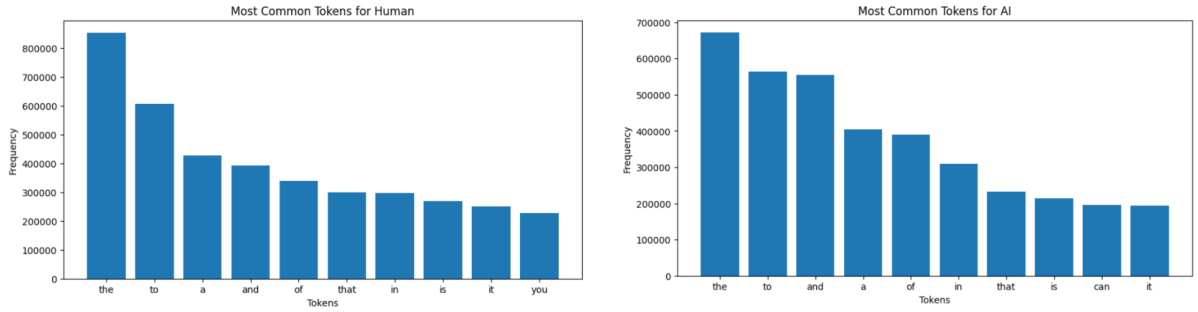


FIGURE 3.7 – Comparaison des tokens les plus fréquents pour les textes humains et IA

Les graphiques présentent les tokens (mots) les plus fréquents pour les textes écrits par des humains et ceux générés par une IA.
On observe que :

- Les mots les plus fréquents sont quasiment identiques dans les deux classes.
- Il s'agit principalement de mots très courants en anglais comme the, to, and, a, of, in, that, is, etc.
- Les fréquences sont également relativement similaires entre les deux catégories.

L'analyse des mots individuels les plus fréquents ne permet pas de discriminer efficacement les textes humains des textes générés par IA, car les distributions sont très similaires.

Les graphiques présentent les 3-grams (groupes de trois mots) les plus fréquents pour les textes écrits par des humains et ceux générés par une IA.

On observe que :

- Les deux classes partagent plusieurs expressions communes, car elles traitent du même thème.
- Les textes humains utilisent davantage de tournures naturelles et conversationnelles, tandis que les textes IA emploient plus de connecteurs logiques formels et une structure plus académique.
- Lorsque l'on augmente la longueur des n-grams, les différences structurelles deviennent plus visibles et la distinction entre les deux classes s'améliore.

mais :

- Les n-grams restent des caractéristiques purement syntaxiques et ne capturent pas la sémantique du texte.
- Les mots fréquents sont souvent communs aux deux classes, donc peu discriminants.
- Construire un modèle uniquement sur les n-grams peut conduire au surapprentissage.

Même si avoir un vocabulaire plus large aide à différencier les classes, ce n'est pas suffisant. Il faut utiliser des modèles de machine learning, comme le deep learning, pour pouvoir comprendre des relations plus complexes entre les textes.

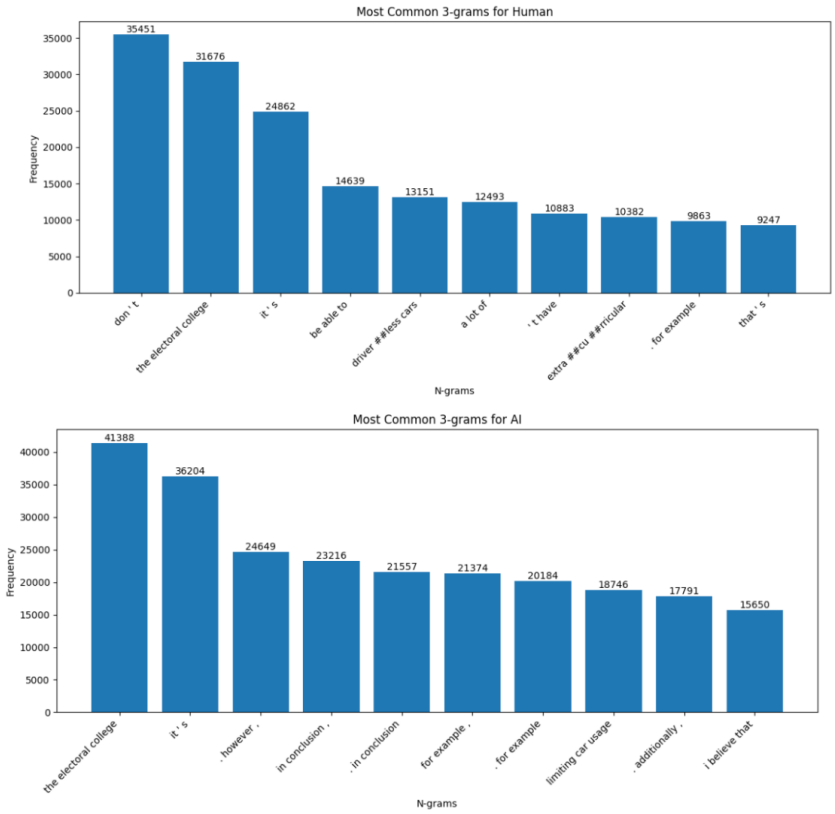


FIGURE 3.8 – Comparaison des 3-gramms les plus fréquents pour les textes humains et IA

3.4 Préparation des Donneés

Après l'analyse des n-grams et leurs limites, il est nécessaire de préparer les données pour un modèle de deep learning. En effet, ces modèles ne comprennent pas le texte brut ; il faut d'abord le transformer en nombres.

Cette transformation se fait avec un tokenizer. Le tokenizer découpe la phrase en petits éléments appelés tokens. Avec BERT, certains mots peuvent être divisés en sous-mots. Ensuite, chaque token est converti en un numéro (input id). Ces numéros sont transformés en tenseurs pour pouvoir être utilisés par PyTorch.

Une classe appelée TextDataset est ensuite créée pour organiser les données. Elle encode les textes, ajoute du padding ou coupe le texte pour que toutes les phrases aient la même longueur, et associe chaque texte à son label (0 pour humain, 1 pour IA).

Enfin, les données sont séparées en trois parties : entraînement, validation et test. Cela permet d'entraîner le modèle et d'évaluer correctement ses performances. Cette méthode est plus efficace que l'utilisation des n-grams seuls, car le deep learning peut apprendre des relations plus complexes dans le texte.

3.5 Conclusion

Ce chapitre a présenté la compréhension des données pour notre projet de détection des textes générés par IA. Nous avons décrit le dataset, analysé la répartition des classes et les tendances lexicales, et préparé les données pour le deep learning via la tokenization et la création de TextDataset. Ces étapes posent les bases pour l'entraînement et l'évaluation du modèle dans les prochaines phases du projet.

Chapitre 4

Modélisation

4.1 Modéle 1 : Long Short-Term Memory (LSTM)

4.1.1 Définition

Long Short-Term Memory (LSTM) est un type spécial de réseau de neurones. Il a été créé pour pallier les faiblesses des réseaux de neurones récurrents (RNN) standards, comme le souci des gradients qui s'estompent. Les LSTM, à la différence des RNN simples, apprennent mieux les liens à long terme. Ils sont super utiles pour comprendre le langage humain et prévoir les événements futurs.

4.1.2 Architecture

L'architecture des LSTMs se base sur un principe clé : les cellules LSTM. Ces cellules gèrent le flux d'informations au moyen de trois portes de la mémoire LSTM importantes. On a la porte d'oubli, la porte d'entrée et la porte de sortie. Les fonctions d'activation sigmoïdales permettent à ces portes de fonctionner de manière optimale.

Ces composantes LSTM jouent un rôle crucial. Elles préservent les informations essentielles sur le long terme. En même temps, elles préviennent les problèmes de gradient explosif ou disparu.

• Porte d'oubli : Décide des informations à garder ou à effacer. Un score près de 0 signifie oublier, près de 1 signifie conserver.
• Porte d'entrée : Juge l'importance des nouvelles informations. Une note près de 0 indique peu d'importance, proche de 1 indique beaucoup d'importance.
• Porte de sortie : Gère combien d'informations vont vers l'état caché suivant. o proche de 0, peu transmis ; proche de 1, beaucoup est transmis.

Grâce à cette structure avancée, l'architecture des LSTMs surpasse les limites des RNN traditionnels. Cela concerne surtout la gestion des dépendances à long terme. Les cellules LSTM et leurs portes complexes aident à relever divers défis.

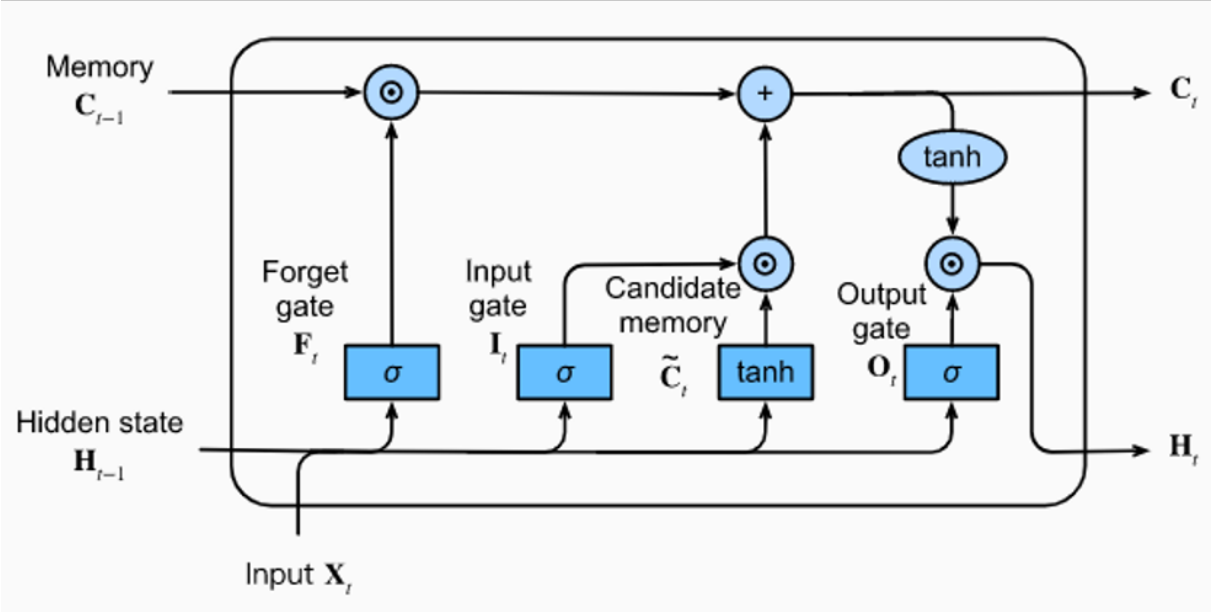


FIGURE 4.1 – Architecture du modéle LSTM

4.1.3 Fonctionnement

Le fonctionnement d'une cellule LSTM se déroule en plusieurs étapes :

• Décider quoi oublier : La porte d'oubli analyse l'état précédent C_{t-1} et l'entrée actuelle x_t pour éliminer les informations inutiles.
• La porte d'entrée et la mémoire candidate déterminent quelles nouvelles informations doivent être ajoutées à l'état de cellule.
• Mettre à jour l'état de cellule :

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Combinaison des informations conservées et des nouvelles informations.
• Produire la sortie :

$$h_t = o_t * \tanh(C_t)$$

L'état de cellule filtré par la porte de sortie devient la sortie de la cellule.

4.1.4 Compositions

Le modèle LSTMClassifier est construit en héritant de la classe nn.Module de PyTorch. Il est composé de plusieurs couches :

• Embedding Layer : Transforme les mots d'entrée (représentés par des indices) en vecteurs denses de taille fixe (embedding dim = 100), et Chaque mot unique du vocabulaire (vocab size = 30522) a son propre vecteur.
• LSTM Layer :
Cœur du modèle, capable d'apprendre les dépendances à long terme dans le texte.
La dimension de sortie de chaque LSTM unit est dimension = 32.
La couche LSTM est composée de 32 unités avec 1 couche.

• Dropout Layer : Permet de régulariser le modèle et de réduire le surapprentissage en désactivant aléatoirement 50% des neurones à chaque étape.
• Fully Connected Layer : Une couche linéaire qui transforme la sortie du LSTM en une valeur unique, ensuite passée à une fonction sigmoïde pour obtenir une probabilité comprise entre 0 et 1.

4.2 Modéle 2 : Convolutional Neural Network (CNN)

4.2.1 Définition

Les Convolutional Neural Networks, ou CNN, sont une classe de réseaux de neurones utilisés dans le Deep Learning ou l'apprentissage profond, particulièrement efficaces pour analyser des données visuelles telles que des images et des vidéos. Cependant, ils se révèlent également très performants pour le traitement du texte, car leurs filtres convolutifs permettent de détecter des motifs locaux dans les séquences de mots, similaires à la manière dont ils identifient des formes dans une image.

4.2.2 Architecture

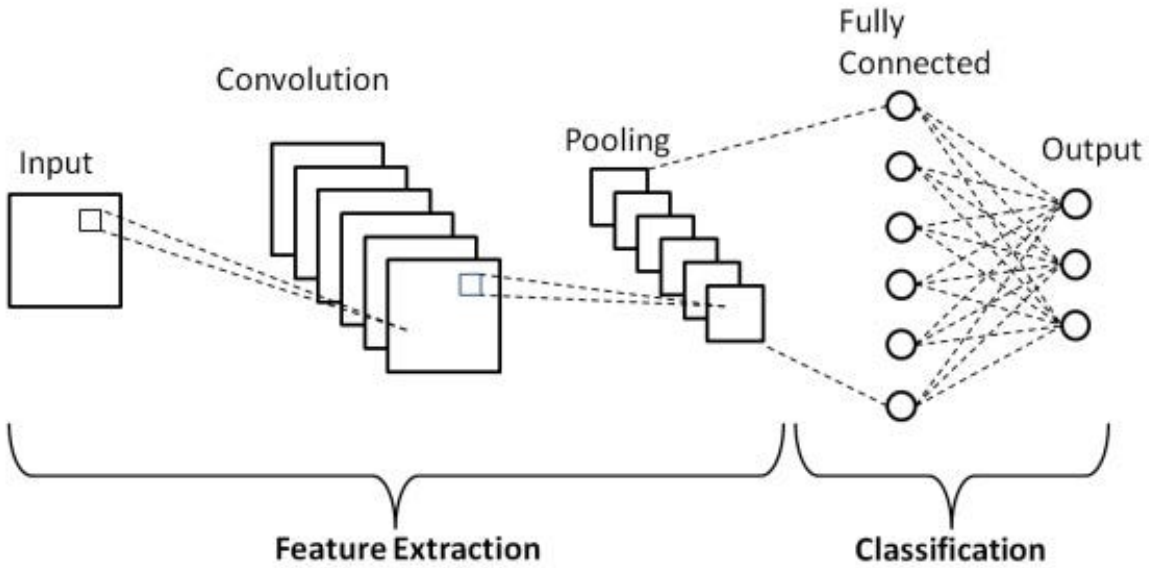


Figure 4.2 – Architecture du modéle CNN

Le modèle CNNClassifier hérite de nn.Module de PyTorch et est composé des couches suivantes :

• Embedding Layer : Transforme chaque mot d'entrée en un vecteur dense de taille embedding_dim = 300. et Les tokens de padding sont mappés à des vecteurs nuls (pad_idx).
• Convolutional Layers (1D) : Plusieurs couches de convolution appliquent des filtres de tailles différentes (filter_sizes = [3,4,5]) sur les embeddings, Chaque filtre détecte un motif spécifique dans le texte et Le nombre de filtres varie selon la taille (num_filters = [100,200,300]).

• Activation ReLU : Introduit la non-linéarité après chaque convolution pour permettre au modèle d'apprendre des motifs complexes.
• Max Pooling : Réduit la dimension des cartes de caractéristiques tout en conservant les informations les plus importantes (valeurs maximales).
• Fully Connected Layer : La sortie des différentes convolutions est concaténée puis passée dans une couche linéaire et Produit un score pour la classification binaire.
• Dropout : Permet de régulariser le modèle en supprimant aléatoirement 50% des neurones avant la couche fully connected.
• Sigmoid Activation : Convertit le score en probabilité entre 0 et 1 pour prédire la classe positive ou négative.

4.2.3 Fonctionnement

Le fonctionnement d'un réseau de neurones convolutif (CNN) se déroule en plusieurs étapes :

• Appliquer la convolution : La couche de convolution analyse l'image d'entrée X à l'aide d'un filtre K afin d'extraire des caractéristiques locales comme les bords ou les textures.
• Calculer la carte de caractéristiques : Le filtre se déplace sur l'image et calcule le produit de convolution pour chaque position.
• Opération de convolution :

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) * K(m,n)$$

Génération d'une carte de caractéristiques représentant les motifs détectés dans l'image.
• Appliquer la fonction d'activation : Une fonction non linéaire comme ReLU est appliquée pour introduire de la non-linéarité dans le réseau.

$$ReLU(x) = \max(0, x)$$

Permet au modèle de mieux apprendre les relations complexes dans les données.
• Réduire la dimension (Pooling) : Une opération de pooling est appliquée pour réduire la taille des cartes de caractéristiques tout en conservant les informations importantes.
• Produire la sortie : Les caractéristiques extraites sont aplaties puis envoyées vers des couches entièrement connectées pour effectuer la classification finale.

4.2.4 Conclusion

Ce chapitre a présenté les concepts fondamentaux des méthodes de deep learning utilisées dans notre projet. Nous avons introduit les architectures principales telles que les réseaux de neurones convolutifs (CNN) et les réseaux récurrents (LSTM), en expliquant leur fonctionnement et leur rôle dans l'extraction et l'apprentissage des caractéristiques à partir des données. Cette étude théorique constitue une base essentielle pour la mise en œuvre et l'entraînement des modèles qui seront utilisés dans les prochaines étapes du projet.

Chapitre 5

Évaluation du modèle

5.1 Introduction

Dans ce chapitre, nous présentons l'évaluation des deux modèles proposés pour la détection des textes générés par intelligence artificielle : un modèle LSTM (*Long Short-Term Memory*) et un modèle CNN (*Convolutional Neural Network*). L'objectif est de mesurer les performances de chacun à l'aide de différentes métriques et d'analyser les résultats obtenus.

5.2 Métriques d'évaluation

Pour évaluer les performances des modèles, plusieurs métriques complémentaires sont utilisées.

L'accuracy (précision globale) mesure la proportion de prédictions correctes parmi l'ensemble des prédictions réalisées :

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

La précision (*Precision*) indique la proportion de textes correctement identifiés comme générés par IA parmi tous ceux prédits comme tels, permettant de mesurer le taux de faux positifs :

$$\text{Precision} = \frac{TP}{TP + FP}$$

Le rappel (*Recall*) mesure la capacité du modèle à détecter l'ensemble des textes générés par IA présents dans le jeu de données :

$$\text{Recall} = \frac{TP}{TP + FN}$$

Le F1-score combine la précision et le rappel afin de fournir une mesure équilibrée des performances, particulièrement utile en cas de déséquilibre entre les classes :

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

où TP, TN, FP et FN désignent respectivement les vrais positifs, vrais négatifs, faux positifs et faux négatifs.

5.3 Résultats expérimentaux

Les deux modèles ont été entraînés sur 75 000 exemples, validés sur 10 000 exemples et évalués sur 15 000 exemples. L'entraînement a été effectué sur 5 époques avec l'optimiseur Adam et la fonction de perte *Binary Cross-Entropy* (`BCELoss`).

5.3.1 Résultats du modèle LSTM

Le modèle LSTM est composé d'une couche d'embedding, d'une couche LSTM unidirectionnelle, d'une couche de dropout ($p = 0{,}5$) et d'une couche fully connected. Le tableau 5.3.1 présente les résultats obtenus sur le jeu de test.

Classe	**Précision**	**Rappel**	**F1-score**	**Support**
Humain (0)	0.95	0.97	0.96	7 450
IA (1)	0.97	0.95	0.96	7 550
Accuracy globale			**0.96**	
Macro avg	0.96	0.96	0.96	15 000
Weighted avg	0.96	0.96	0.96	15 000

Table 5.1 – Rapport de classification – Modèle LSTM

La perte sur le jeu de test (*Test Loss*) s'établit à **0.127**, et l'accuracy de test atteint **96%**. La matrice de confusion du modèle LSTM est représentée à la figure **??**.

5.3.2 Résultats du modèle CNN

Le modèle CNN est composé d'une couche d'embedding, de trois couches convolutionnelles 1D avec des filtres de tailles $\{3, 4, 5\}$ et de nombres de filtres $\{100, 200, 300\}$, d'une opération de max-pooling, d'une couche de dropout, et d'une couche fully connected. Le tableau 5.3.2 présente les résultats obtenus.

Classe	**Précision**	**Rappel**	**F1-score**	**Support**
Humain (0)	1.00	1.00	1.00	7 450
IA (1)	1.00	1.00	1.00	7 550
Accuracy globale			**0.998**	
Macro avg	1.00	1.00	1.00	15 000
Weighted avg	1.00	1.00	1.00	15 000

Table 5.2 – Rapport de classification – Modèle CNN

La perte sur le jeu de test s'établit à **0.009**, et l'accuracy de test atteint **99,8%**. La matrice de confusion du modèle CNN est représentée à la figure **??**.

5.3.3 Comparaison des deux modèles

Le tableau 5.3.3 synthétise les principales métriques des deux modèles.

Métrique	LSTM	CNN
Test Accuracy	96.0%	99.8%
Test Loss	0.127	0.009
F1-score moyen	0.96	1.00
Faux positifs	Modéré	Très faible
Faux négatifs	Modéré	Très faible

Table 5.3 – Comparaison des performances LSTM vs CNN

5.4 Analyse des résultats

5.4.1 Analyse du modèle LSTM

Le modèle LSTM obtient une accuracy de **96%** avec une perte de test de 0.127. Les métriques sont équilibrées entre les deux classes, ce qui indique l'absence de biais significatif. La matrice de confusion révèle légèrement plus de faux négatifs que de faux positifs, comportement acceptable dans un contexte académique où les faux positifs (accusations injustifiées) sont plus problématiques. La troncature à 512 tokens imposée par le tokeniseur BERT constitue la principale limite de ce modèle pour les textes longs.

5.4.2 Analyse du modèle CNN

Le modèle CNN atteint une accuracy de **99,8%** avec une perte de test inférieure à 0.01, surpassant largement le LSTM. Cette performance s'explique par la capacité des filtres convolutionnels de tailles variées (3, 4 et 5 mots) à capturer les motifs stylistiques et lexicaux caractéristiques des textes générés par IA. La convergence dès la première époque confirme que ces motifs sont saillants et facilement détectables. Ces résultats sont néanmoins à nuancer : en conditions réelles, avec des textes plus variés ou issus de modèles de génération plus récents, les performances pourraient être inférieures.

5.4.3 Comparaison de deux modéles

Le CNN surpasse le LSTM pour trois raisons principales : la tâche repose davantage sur la reconnaissance de motifs locaux que sur la compréhension séquentielle globale ; le CNN converge plus rapidement ; et il est moins affecté par la troncature à 512 tokens, car il opère sur des fenêtres locales.

5.5 Conclusion

Ce chapitre a comparé les performances du LSTM et du CNN pour la détection de textes générés par IA. Le CNN s'est révélé nettement supérieur (99,8% contre 96% d'accuracy, perte quatorze fois inférieure), confirmant l'adéquation des architectures convolutionnelles à cette tâche.

Chapitre 6

Déploiement

6.1 Introduction

Ce chapitre présente le déploiement de l'application de détection de textes générés par intelligence artificielle. L'application est composée d'un backend FastAPI et d'un frontend React, tous deux déployés sur la plateforme Render.

6.2 Technologies utilisées

6.2.1 FastAPI

FastAPI est un framework Python utilisé pour développer l'API REST de l'application. Il expose deux routes principales décrites dans le tableau 6.1.

Méthode	**Route**	**Description**
GET	`/`	Vérification du statut de l'API
POST	`/predict`	Classification d'un texte soumis

Table 6.1 – Routes de l'API FastAPI

6.2.2 React et Vite

Le frontend est développé avec React et Vite. Il permet à l'utilisateur de saisir un texte, de lancer l'analyse et de consulter le résultat de la classification ainsi que l'historique des analyses.

6.2.3 Render

Render est une plateforme cloud gratuite permettant d'héberger des applications web. Elle a été choisie pour sa simplicité de déploiement depuis GitHub, son support natif de Python et de Node.js, et la génération automatique d'un certificat HTTPS.

6.2.4 Git et GitHub

Git est utilisé pour le versionnement du code source et GitHub comme dépôt distant. Render se connecte au dépôt GitHub et déclenche automatiquement un nouveau

déploiement à chaque push.

6.3 Déploiement sur Render

6.3.1 Étapes de déploiement

Le déploiement du backend et du frontend s'effectue en suivant les étapes suivantes :

1. Le code source complet (backend et frontend) est poussé vers le dépôt GitHub.
2. Un premier Web Service est créé sur Render pour le **backend** avec les paramètres : *Root Directory* = `backend`, *Build Command* = `pip install -r requirements.txt`, *Start Command* = `uvicorn main:app --host 0.0.0.0 --port $PORT`.
3. Un second Web Service est créé sur Render pour le **frontend** avec les paramètres : *Root Directory* = `frontend`, *Build Command* = `npm install && npm run build`, *Publish Directory* = `dist`.
4. Render installe automatiquement les dépendances, compile les fichiers et démarre les deux services.
5. L'URL publique du backend est renseignée dans le frontend pour que les appels API pointent vers le service déployé.

6.3.2 Résultat

Une fois déployés, les deux services sont accessibles aux adresses suivantes :
— **Frontend** : `https://ai-text-detector.onrender.com`
— **API** : `https://ai-text-detector-api.onrender.com`
— **Documentation** : `https://ai-text-detector-api.onrender.com/docs`

6.4 Tâches réalisées

Les figures suivantes illustrent les deux cas de classification de l'application. La figure 6.1 montre un texte classifié comme humain et la figure 6.2 montre un texte classifié comme généré par IA.

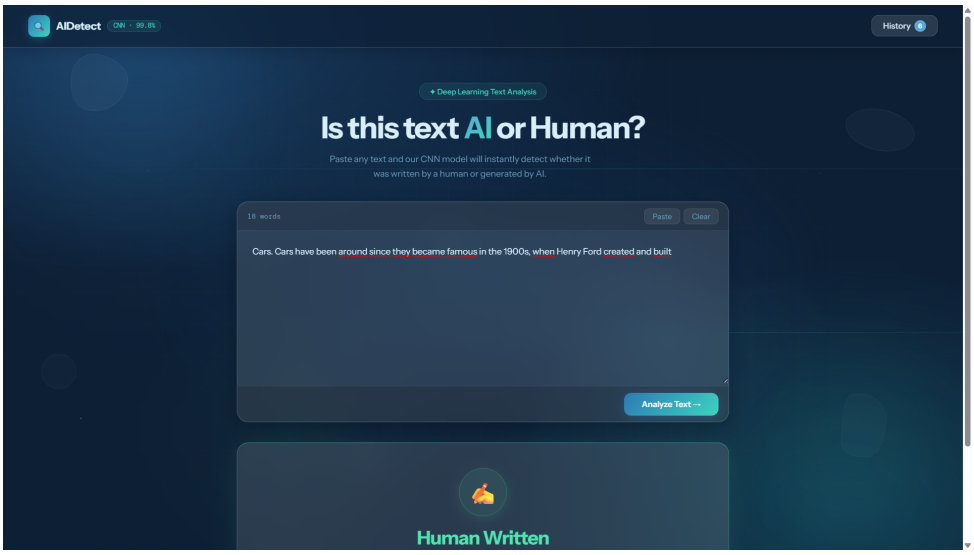


FIGURE 6.1 – Résultat *Human Written* — texte classifié comme rédigé par un humain.

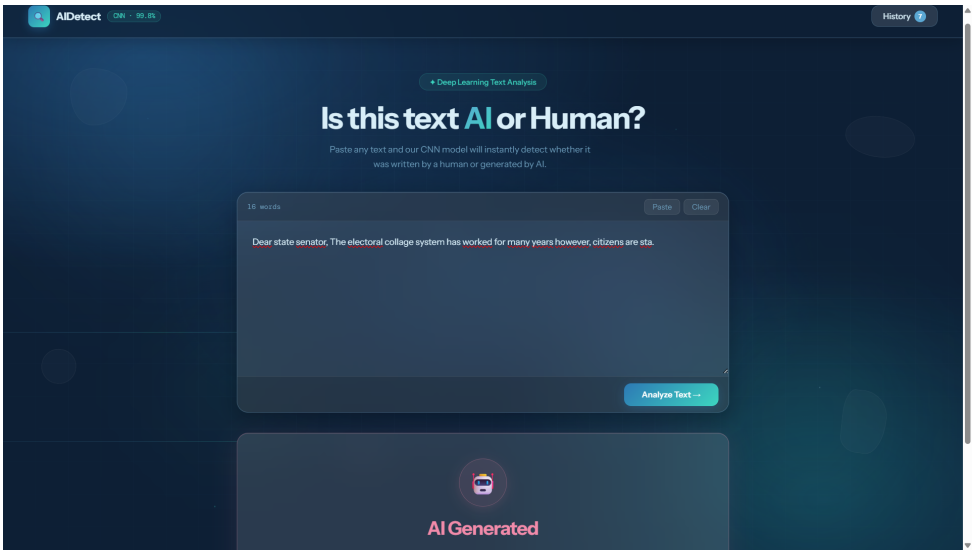


FIGURE 6.2 – Résultat *AI Generated* — texte classifié comme généré par une IA.

6.5 Conclusion

Ce chapitre a présenté le déploiement complet de l'application sur la plateforme Render, aussi bien le backend FastAPI que le frontend React. Cette architecture entièrement hébergée en ligne permet une utilisation accessible depuis n'importe quel navigateur, sans nécessiter d'installation locale. La synchronisation entre GitHub et Render assure un redéploiement automatique à chaque mise à jour du code source.

Conclusion Générale

Ce projet a été consacré à la conception et au développement d'un système de détection des textes générés par l'intelligence artificielle, dans un contexte où la distinction entre productions humaines et contenus automatiques devient un enjeu croissant dans les domaines académiques, professionnels et médiatiques. En suivant la méthodologie CRISP-DM, nous avons mené chaque étape du projet de manière structurée : de la compréhension de la problématique jusqu'au déploiement de l'application, en passant par la collecte et la préparation d'un jeu de données de 487 235 textes, la modélisation et l'évaluation de deux architectures de deep learning. Le prototype développé démontre la faisabilité technique de notre approche et illustre la pertinence d'un tel outil pour détecter automatiquement l'origine d'un texte.

Les résultats obtenus confirment la supériorité du modèle CNN, qui atteint une accuracy de 99,8% contre 96% pour le modèle LSTM, grâce à sa capacité à identifier des motifs stylistiques et lexicaux caractéristiques des textes générés par IA. Ces performances ont été rendues possibles par une préparation rigoureuse des données, notamment le rééquilibrage des classes et la tokenisation via BERT. Toutefois, des limites subsistent : la sensibilité du modèle à la qualité du jeu de données, la difficulté à généraliser à des textes produits par des modèles génératifs plus récents, et la nécessité d'améliorer l'interprétabilité des prédictions. Ces constats ouvrent la voie à des perspectives futures, notamment l'adoption d'architectures Transformer pré-entraînées, l'enrichissement du corpus d'entraînement et l'adaptation à différents contextes linguistiques.

En définitive, ce travail constitue une première étape vers la mise en place d'outils fiables permettant de mieux encadrer l'usage de l'intelligence artificielle dans la production écrite. L'application déployée sur Render, combinant un backend FastAPI et un frontend React, offre une solution accessible et fonctionnelle. Il met en évidence l'importance de continuer à explorer ce domaine en constante évolution, afin d'apporter des solutions toujours plus robustes et adaptées aux défis posés par les technologies émergentes.